

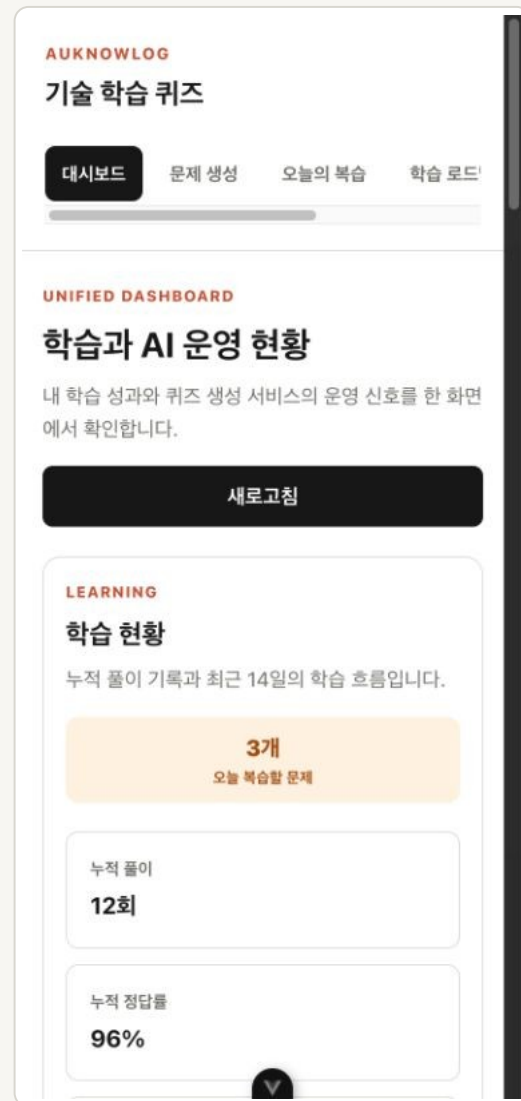
Auknowlog

기술 공부를 기사·문제·복습까지
이어가기 위해 만든 개인 학습 도구

처음에는 주제별 퀴즈를 만드는 작은 프로젝트였습니다.
직접 사용하며 핵심 내용 누락, 중복 문제, AI 호출 관측과 데일리 학습
흐름을 서버 정책으로 보완했습니다.

Java 21 · Spring Boot · PostgreSQL · pgvector · Vue 3

개인 프로젝트 2025.08 - 현재



실제 모바일 화면

처음에는 문제를 만드는 기능 하나뿐이었습니다

직접 써 보면서 학습이 끝나는 지점을 하나씩 고쳤습니다.

2025.08 · 시작

Gemini로 주제별 퀴즈를 만들고 화면에 보여주는 단일 기능

첫 커밋에는 QuizController, GeminiService와 하나의 Vue 화면만 있었습니다.

사용하면서 발견한 문제

01 핵심 내용이 빠질 수 있었습니다.

주제명만 전달하는 방식은 필수 학습 범위를 보장하기 어려웠습니다.

02 비슷한 문제가 반복됐습니다.

문자열 비교만으로 표현이 다른 중복을 찾기 어려웠습니다.

03 비용과 품질 저하 지점이 보이지 않았습니다.

호출 결과만으로는 토큰과 중복 제거 과정을 추적할 수 없었습니다.

기능을 늘릴 때마다 “이 기능이 다음 학습 행동으로 이어지는가”를 기준으로 삼았습니다.

현재 학습 흐름

자료나 주제로 계획을 세우고, 문제를 푼 뒤 복습과 다음 학습으로 이어집니다.

1 주제·자료 또는 RSS 기사

직접 입력한 주제와 자료, 아침 기술 기사를 학습 출발점으로 사용합니다.

2 로드맵·해설과 문제 생성

필수 목표와 기사 해설을 바탕으로 중복을 거른 문제를 준비합니다.

3 서버 채점과 기록

서버가 정답을 비교하고 문항별 답안과 복습 일정을 남깁니다.

4 복습과 다음 추천

오답 일정, 목표 진행률과 정답률을 다음 학습에 사용합니다.

AUKNOWLOG
기술 학습 퀴즈

대시보드 문제 생성 오늘의 복습 학습 로드맵 학습 자료 풀이 기록

QUIZ
새 퀴즈 만들기

학습할 주제와 문제 수를 정하고, 필요하면 참고 자료를 함께 입력하세요.

주제:
예: 자바스크립트, 인공지능

문제 수 (기본 5, 최대 20):
5

학습 자료 제목 (선택):
예: JVM 실행 구조

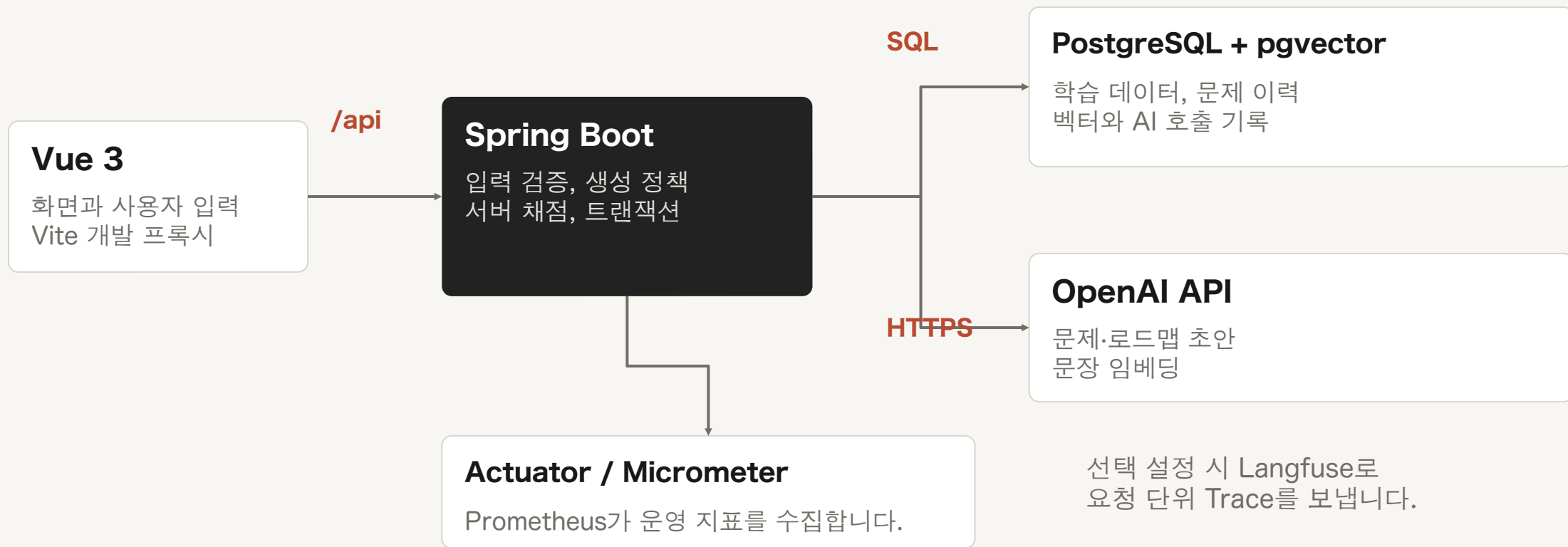
학습 자료 내용 (선택):
Markdown 또는 기술 문서 내용을 붙여넣으면 퀴즈와 함께 저장합니다.

실제 문제 생성 화면

주요 메뉴: 대시보드, 데일리 학습, 문제 생성, 오늘의 복습, 학습 로드맵, 학습 자료, 풀이 기록, 품질 평가

전체 구조

서버가 채점과 저장의 기준이 되고, AI와 관측 도구는 필요할 때만 연결합니다.



현재 PostgreSQL과 Prometheus만 Docker Compose로 실행합니다. 프론트엔드와 백엔드는 로컬 프로세스이며 Kubernetes와 Redis는 사용하지 않습니다.

정답을 브라우저로 보내지 않도록 API 계약을 바꿨습니다

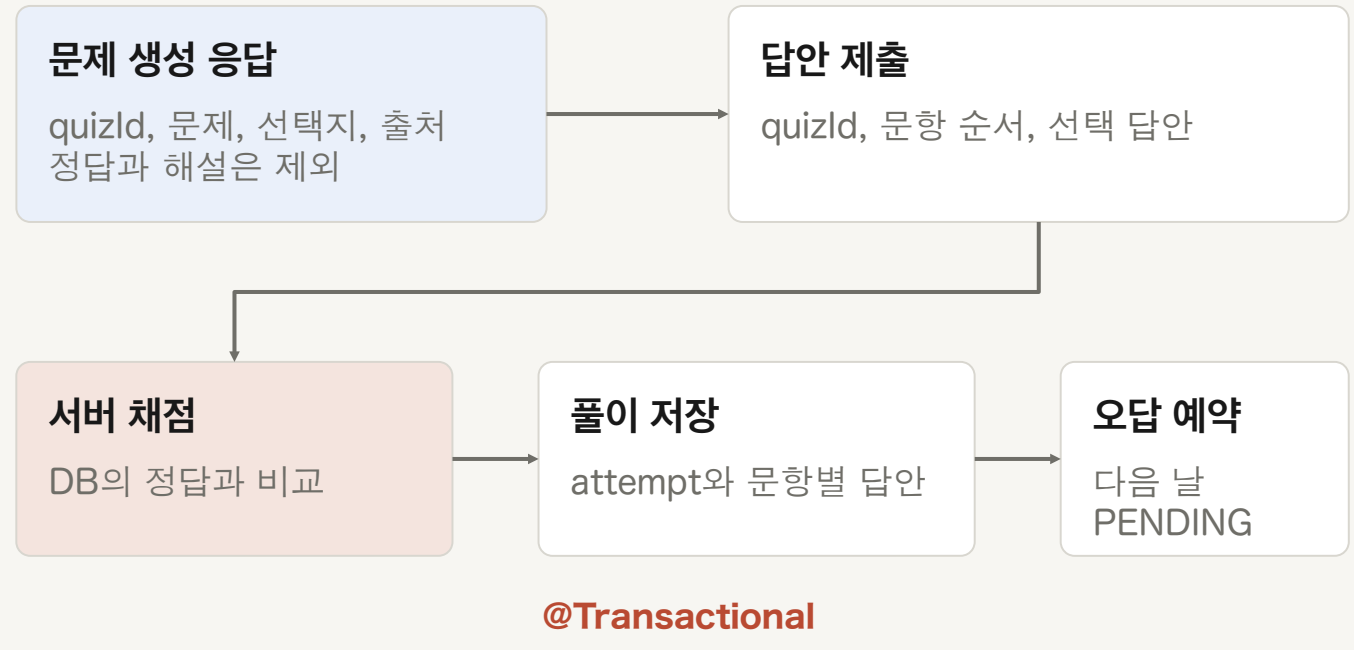
UI에서 감추는 방식으로는 정답을 보호할 수 없었습니다.

변경 전

문제 생성 응답에 정답과 해설까지 포함

개발자 도구의 Network 응답을 보면
정답을 확인할 수 있었습니다.

변경 후



채점, 풀이 저장과 오답 복습 예약을 한 DB 트랜잭션으로 처리합니다. 중간에 실패하면 함께 롤백합니다.

같은 문제와 비슷한 문제를 나눠서 처리했습니다

비용이 거의 없는 해시 검사를 먼저 하고, 표현이 다른 경우에만 벡터 유사도를 봅니다.

정확히 같은 문제

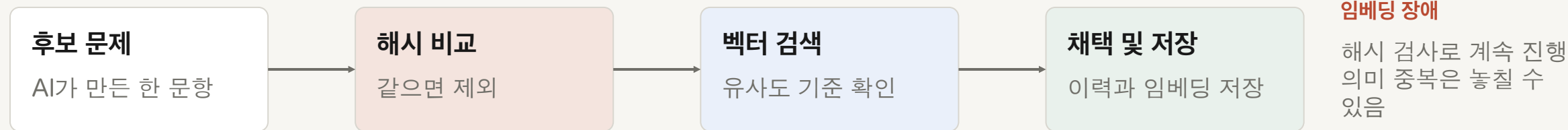
정규화 + SHA-256

공백과 대소문자를 정리한 뒤 해시를 비교합니다.
C++는 cplusplus, C#은 csharp로 바꿔 기술명 충돌을 피했습니다.

표현이 다른 비슷한 문제

512차원 임베딩 + 코사인 유사도

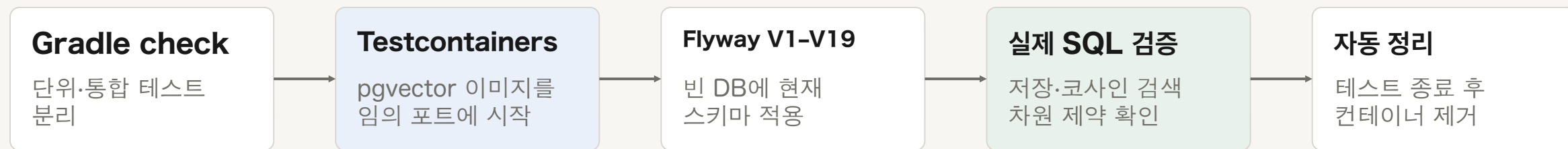
문장을 숫자 벡터로 바꾸고 pgvector에서 가까운 문제를 찾습니다.
일반 중복 기준은 0.90, 반복 피드백 문항은 0.82입니다.



HNSW 인덱스는 가까운 벡터 후보를 찾는 데 사용합니다. 42쌍 기준 데이터셋과 사람 검토로 임계값을 평가하며, 대규모 성능은 아직 측정하지 않았습니다.

pgvector는 실제 PostgreSQL에서 검증했습니다

H2가 흉내 낼 수 없는 확장 타입과 거리 연산을 일회용 DB에서 실행합니다.



DB 제약

vector 확장, vector(512), HNSW와 코사인 연산자 구성을 확인했습니다. 511차원 저장은 DB가 거부했습니다.

중복 판정

가까운 벡터는 0.90에서 차단하고 직교 벡터는 통과시켰습니다. 피드백 기준 0.82도 별도로 확인했습니다.

검증 경계

고정 벡터를 써서 OpenAI 호출은 0 회입니다. 임베딩 장애 시 해시 풀백은 서비스 단위 테스트로 책임을 분리했습니다.

이 테스트는 DB 호환성과 애플리케이션 SQL을 증명합니다. 실제 한국어 임베딩 품질과 HNSW 대용량 성능은 별도 평가가 필요합니다.

보충 생성도 목표와 비용 상한 안에서 처리했습니다

중복으로 부족해진 문항만 보충하며, 로드맵에서는 남은 학습 목표 배정을 유지합니다.



반복 횟수를 제한했습니다

보충은 전체 3회까지만 진행합니다. 계속 중복되면 목표 개수보다 적은 문제를 반환할 수 있습니다.

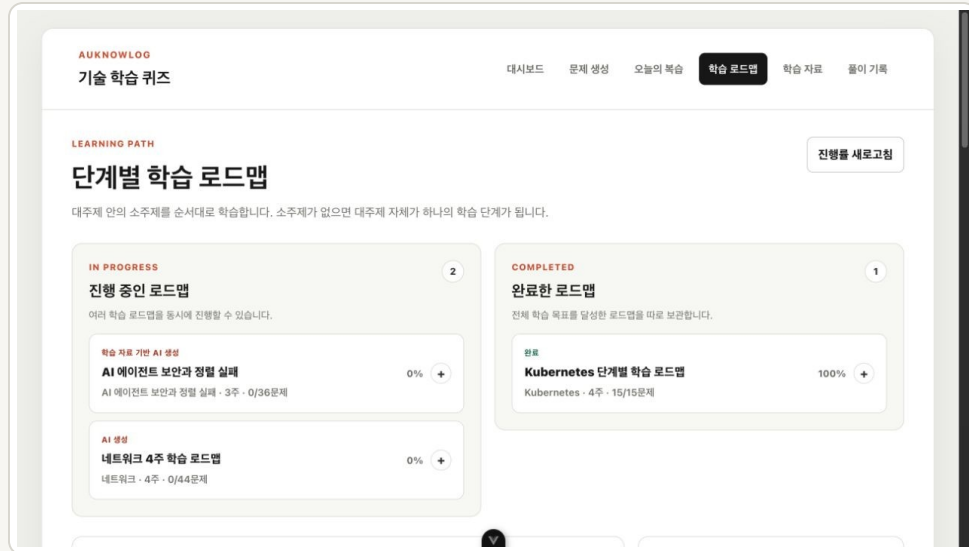
목표 배정을 유지했습니다

목표 기반 로드맵은 통과한 문항의 objectiveKey를 빼고, 부족한 목표만 다시 요청합니다.

보충은 최대 3회, 한 호출은 최대 20문제입니다. 첫 요청의 최근 피드백 문항 미리보기는 최대 30개로 제한하며 모델은 GPT-5.4 mini / low를 사용합니다.

필수 학습 목표가 문제의 출제 범위를 결정합니다

주제명만 전달할 때 생기던 핵심 내용 누락을 목표별 커버리지로 줄였습니다.



진행 중·완료 로드맵

저장 전 미리보기

AI가 만든 단계와 목표, 순서와 문제 수를 수정합니다.
최종 저장은 모델을 다시 호출하지 않습니다.



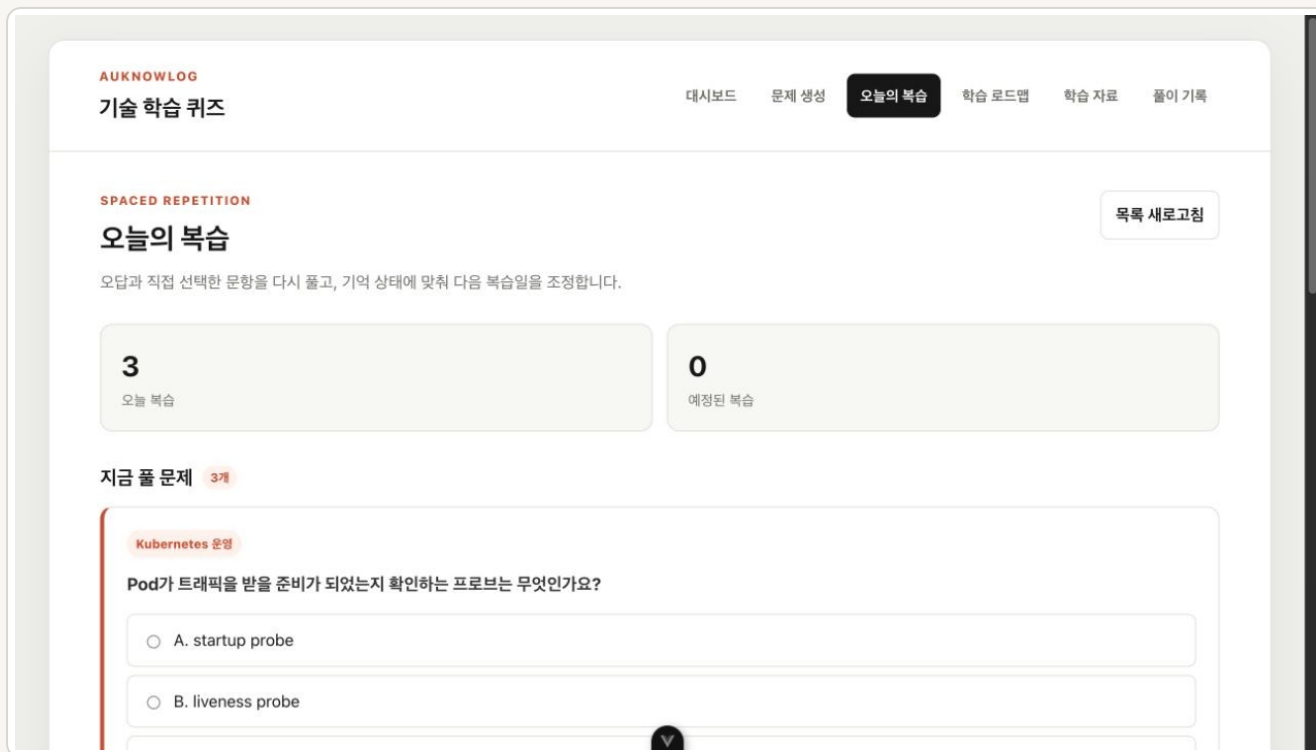
풀이 결과가 다음 배정의 커버리지로 다시 반영됩니다.

연결 자료가 있으면 로드맵과 문제 생성 모두 같은 자료를 사용합니다.

학습 단위는 최대 30문제를 누적하고, 한 번에는 20문제까지만 만듭니다. 목표가 없는 v1.0/v1.1 로드맵은 기존 주제 방식으로 동작합니다.

오답이 다음 복습으로 이어지도록 했습니다

한 번 생성한 문제를 다시 사용하므로 복습과 일정 계산에는 AI 비용이 들지 않습니다.



실제 복습 화면

복습 대상

오답은 자동으로 넣습니다. 맞힌 문제도 필요하면 직접 복습에 추가할 수 있습니다.

간격 조정

연속으로 맞히면 3일, 7일, 14일, 30일 순서로 간격을 늘립니다. 5회 연속 정답이면 완료합니다.

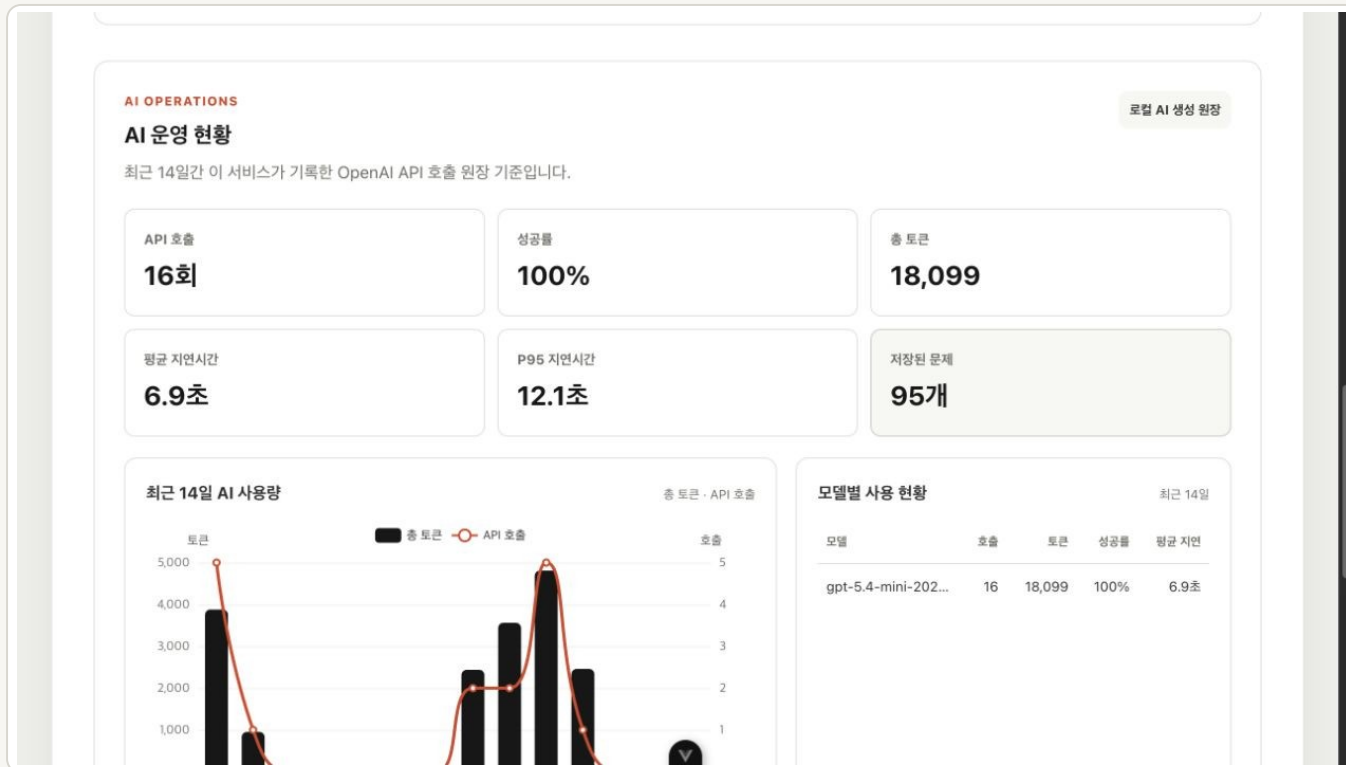
중복 방지

같은 문제에는 PENDING 일정을 하나만 두고, 제출 UUID로 동일 요청의 재처리를 막습니다.

추천은 주제별 정답률과 마지막 학습일을 이용한 규칙 기반 방식입니다. 적은 데이터에서도 결과와 추천 이유를 확인할 수 있도록 AI를 사용하지 않았습니다.

AI 호출 원장은 PostgreSQL에 남겼습니다

서버 재시작과 수집 중단 뒤에도 요청별 사용량과 실패 기록이 남아야 했습니다.



2026-09-13 로컬 DB의 최근 14일 기록

화면의 16회 호출과 18,099토큰은 로컬 사용 기록입니다. 부하 처리량이나 비용 절감 효과를 나타내는 수치는 아닙니다.

요청별 원장

operation, model, 성공 여부, 입력·출력·총 토큰과 지연시간을 ai_generation_log에 저장합니다.

학습 지표와 함께 조회

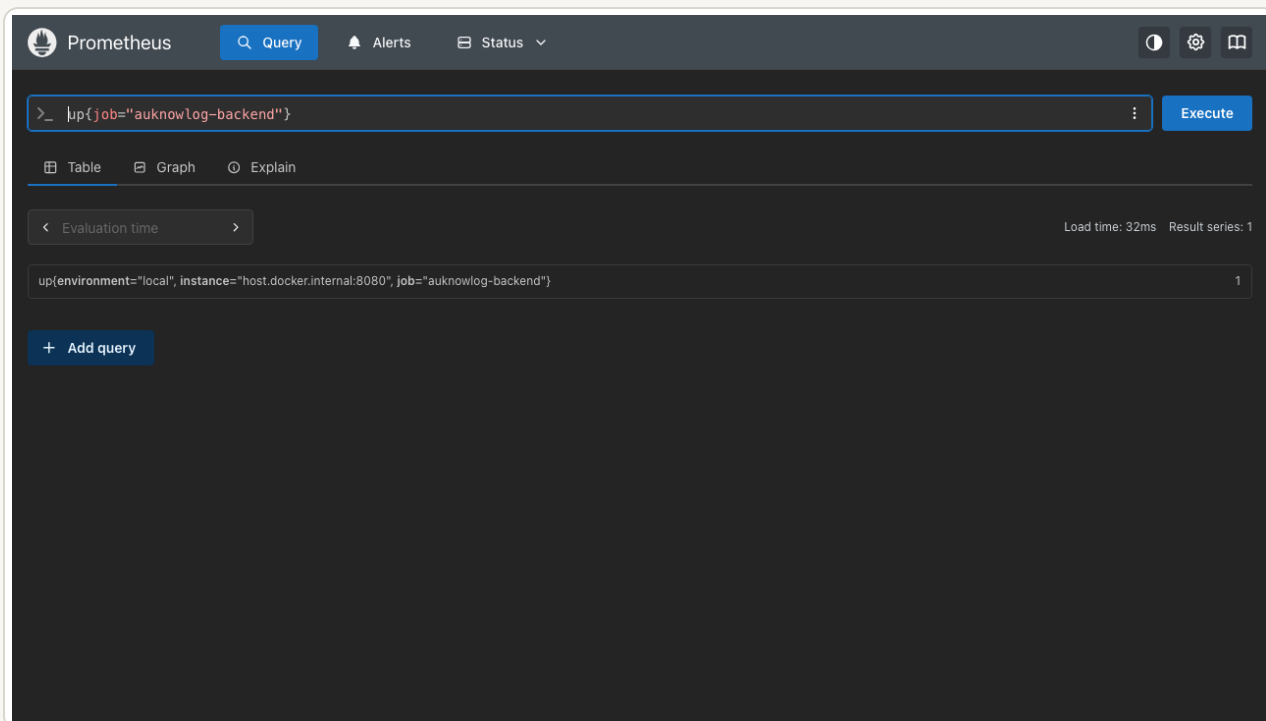
같은 기간의 풀이·복습 정보와 AI 사용량을 나눠 보여줍니다. 금액 대신 토큰 원본을 표시합니다.

Trace는 선택 사항

Langfuse는 요청 내부 단계가 필요할 때만 켵니다. 키가 없어도 핵심 기능은 동작합니다.

Prometheus로 비용과 문제 품질 신호를 분리했습니다

호출량보다 “한 번의 생성 안에서 어디서 비용과 품질이 나빠졌는가”를 봅니다.



2026-09-14 로컬 Prometheus · `up{job="auknowlog-backend"} = 1`

Prometheus는 선택 실행이며 15초 pull·60일 보관입니다. 저사용량에서는 5분 비율 대신 7일 절대 건수를 먼저 보고, Grafana·알림은 아직 도입하지 않았습니다.

관측하는 값

AI 호출 시간·토큰, 전체 생성 시간, 생성 시도 수, 채택·정확 중복·의미 중복 문항 수를 분리했습니다.

label 경계

operation, model, 제한된 outcome만 사용합니다.
topic, 문제 본문과 요청 ID는 넣지 않아 시계열
폭증과 내용 노출을 막았습니다.

재현 가능한 검증

계약 테스트는 실제 export 문자열을 확인하고,
스크립트는 readiness와 scrape target의 UP 상태를
확인합니다.

학습 흐름에 맞춘 데이터 구조

테이블을 기능 단위로 나누고, 복습과 중복 검사에 필요한 제약을 DB에 두었습니다.

학습 자료	source_document	source_chunk	원문 해시로 중복을 확인하고 본문을 청크로 나눕니다.
로드맵	learning_roadmap / step	learning_objective	단계 순서와 CORE·SUPPORTING 목표별 문제 수를 저장합니다.
퀴즈 풀이	learning_quiz / question	learning_attempt / answer	문항을 학습 목표 FK로 연결하고 문항별 선택 답안을 남깁니다.
복습	review_schedule	review_attempt	다음 복습일과 상태, 재풀이 결과와 변경 전후 간격을 기록합니다.
AI 품질·데일리	question_history / embedding	quality·daily_learning	문제 해시·벡터·사람 판정과 날짜별 기사 학습을 분리해 관리합니다.

V13은 학습 목표 FK, V14~V15는 품질 평가, V16~V18은 단계 결정·보기별 해설·데일리 학습, V19는 기사 선택 등급을 추가했습니다.

기술 선택과 현재 적용 범위

프로젝트 규모에 필요한 만큼 사용하고, 아직 필요하지 않은 기술은 넣지 않았습니다.

선택	왜 선택했는가	현재 적용 범위
Spring Boot / Java 21	채점과 복습처럼 여러 저장 작업을 트랜잭션으로 묶어야 했습니다.	HTTP 계층과 유스케이스를 분리하고 서버 검증을 서비스에 모았습니다.
PostgreSQL + pgvector	관계 데이터와 벡터를 한 DB에서 관리해 Elasticsearch 이중 운영을 줄였습니다.	JPA는 학습 데이터에, JdbcTemplate은 벡터 거리 SQL에 사용합니다.
Flyway	자동 DDL 대신 검토 가능한 SQL과 재현 가능한 스키마가 필요했습니다.	V1-V19 마이그레이션을 적용하고 Hibernate는 스키마를 검증합니다.
Testcontainers	H2에서는 vector 타입과 HNSW, 코사인 거리 연산을 검증할 수 없습니다.	실제 pgvector DB 테스트에서 스키마와 유사 벡터 검색을 확인합니다.
Vitest / Playwright	빌드만으로는 제출 상태와 화면 전환, 모바일 회귀를 확인할 수 없습니다.	고정 fixture로 Chromium E2E를 실행하고 Node 22 CI에서 실패 trace를 보관합니다.
Docker Compose	개인 로컬 사용에 Kubernetes 운영 비용은 기능보다 컸습니다.	PostgreSQL과 선택형 Prometheus만 컨테이너로 실행합니다.

검증 근거와 다음 과제

확인한 사실과 아직 측정하지 않은 부분을 구분해 두었습니다.

2026-09-28 로컬 검증 경로

서버·브라우저를 분리해 검증

Spring 통합 테스트는 정답 비공개, 서버 채점, 풀이 저장과 복습 예약을 확인합니다.

Testcontainers는 Flyway V1-V19, vector(512), HNSW와 코사인 검색을 실제 DB에서 확인합니다.

브라우저에서 확인

Vitest는 제출 상태를, Playwright Chromium은 생성·제출·데일리 완료와 360px 화면을 고정 fixture로 확인합니다.

CI는 OpenAI, RSS와 개인 DB를 호출하지 않습니다.

다음에 검증할 것

문단 단위 근거 인용

기사·학습 자료의 어느 문단을 근거로 해설과 문제를 만들었는지 표시합니다.

사용자 인증

여러 사용자를 위한 데이터 소유권과 접근 권한을 설계합니다.

전체 컨테이너화

백엔드·프론트엔드 이미지와 새 환경의 한 명령 기동을 검증합니다.

제한적 클라우드 배포

비용 알림을 먼저 둔 Terraform plan과 ECS 배포·시크릿 주입을 검토합니다.

핵심 사례는 학습 목표 기반 출제 통제, pgvector 중복 판정, Prometheus 관측과 비용 없는 브라우저 회귀 검증입니다. 각 선택의 적용 범위와 검증 경계를 함께 설명합니다.